PHP Cheatsheet

Running PHP

```
php -v                    # version
php script.php            # run a script
php -l script.php         # syntax check
php -S localhost:8000     # web server in the current folder
php --ini / php -m        # config file / loaded extensions
```

Basics

```
<?php
declare(strict_types=1);          // optional: strict typing for function calls

// comment   # comment   /* block */
$name = 'Alice';                  // variable
const MAX = 10;                   // constant
echo "Hi $name", PHP_EOL;         // double quotes interpolate
echo 'Hi ' . $name;               // . concatenates
?>
<p><?= $name ?></p>               <!-- short echo in HTML -->
```

Types	Check	Cast
`int` `float` `string` `bool` `array` `object` `null`	`var_dump`, `gettype`, `is_int`…	`(int)`, `(float)`, `(string)`, `(bool)`, `(array)`

Falsy: `false 0 0.0 "" "0" [] null` — Compare with `===` / `!==`.

```
isset($x)      // exists and not null
empty($x)      // not set or falsy
$x ?? 'def'    // default if null/unset
$x ?: 'def'    // default if falsy
$a <=> $b      // -1, 0, 1 (sorting)
```

Control structures

```
if ($a) { } elseif ($b) { } else { }
$v = $cond ? 'yes' : 'no';
$v = match ($x) { 1, 2 => 'low', 3 => 'mid', default => 'high' };

for ($i = 0; $i < 10; $i++) { }
while ($cond) { }
foreach ($arr as $value) { }
foreach ($arr as $key => $value) { }
break; continue;
```

Template syntax in HTML: `<?php if ($x): ?> ... <?php endif; ?>`, `<?php foreach ($a as $v): ?> ... <?php endforeach; ?>`

Arrays

```
$list = ['a', 'b'];            $list[] = 'c';
$map  = ['k' => 'v'];           $map['k2'] = 'v2';
```

`count` `in_array` `array_key_exists` `array_search` `array_keys` `array_values` `array_merge` `array_slice` `array_push/pop` `array_shift/unshift` `array_map` `array_filter` `array_sum` `array_column` `implode` `explode` `max` `min`

Sorting (in place): `sort/rsort` (values, re-index) · `asort/arsort` (values, keep keys) · `ksort/krsort` (keys) · `usort($a, fn($x, $y) => $x <=> $y)`

Functions

```php
function f(int $a, ?string $b = null, int|float ...$rest): string { return ''; }
f(1, b: 'x');                       // named arguments
$fn = fn($x) => $x * 2;             // arrow function
$fn2 = function ($x) use ($y) { };  // closure
```

Strings & dates

`strlen` `mb_strlen` `strtolower/upper` `ucfirst` `trim` `str_contains` `str_starts_with` `str_ends_with` `str_replace` `substr` `strpos` (`=== false`!) `sprintf('%s %d %.2f')` `number_format($n, 2, ',', ' ')` `nl2br` `htmlspecialchars`

```php
date('d/m/Y H:i');  strtotime('+1 day');  time();
$d = new DateTime('2026-12-25');  $d->format('Y-m-d');  $d->modify('+1 week');
(new DateTime())->diff($d)->days;
```

OOP

```php
namespace App\Shop;
use App\Other\Thing;

interface Sellable { public function price(): float; }

abstract class Item { abstract public function name(): string; }

class Product extends Item implements Sellable
{
    public const VAT = 0.2;
    private static int $count = 0;

    public function __construct(
        private string $name,
        protected float $price,
        public readonly string $sku,
    ) { self::$count++; }

    public function name(): string { return $this->name; }
    public function price(): float { return $this->price * (1 + self::VAT); }
}

$p = new Product('Mouse', 10, 'M-1');
$p->price();  Product::VAT;  $p instanceof Sellable;  $copy = clone $p;
```

`public` / `protected` / `private` · `static` · `self::` / `parent::` / `static::` · `final` · `abstract` · `trait` + `use`

```php
try { throw new InvalidArgumentException('msg'); }
catch (InvalidArgumentException $e) { $e->getMessage(); }
finally { }
```

Autoload: `spl_autoload_register(fn($class) => require __DIR__ . '/src/' . str_replace('\\', '/', $class) . '.php');`

Web

```php
$_GET['q'] ?? ''      $_POST['name']     $_FILES['f']     $_SERVER['REQUEST_METHOD']
htmlspecialchars($s, ENT_QUOTES, 'UTF-8')     // ALWAYS when displaying user data
filter_var($email, FILTER_VALIDATE_EMAIL)
header('Location: page.php'); exit;            // before any output!
```

Sessions: `session_start()` (top of page) · `$_SESSION['k'] = $v` · `session_regenerate_id(true)` after login · `session_destroy()`

Cookies: `setcookie('k', 'v', ['expires' => time() + 3600, 'path' => '/', 'httponly' => true, 'samesite' => 'Lax'])` · `$_COOKIE['k']`

Passwords: `password_hash($pw, PASSWORD_DEFAULT)` · `password_verify($pw, $hash)`

Upload: `enctype="multipart/form-data"` · check `error === UPLOAD_ERR_OK`, size, `(new finfo(FILEINFO_MIME_TYPE))->file($tmp)` · random name · `move_uploaded_file($tmp, $dest)`

Files: `file_exists` `is_dir` `mkdir` `file_get_contents` `file_put_contents` `unlink` `glob` `scandir` `basename` `pathinfo`

PDO

```php
$pdo = new PDO('sqlite:' . __DIR__ . '/app.db', null, null, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
// MySQL: 'mysql:host=localhost;dbname=db;charset=utf8mb4', $user, $pass

$rows = $pdo->query('SELECT * FROM t')->fetchAll();          // no user data only
$stmt = $pdo->prepare('SELECT * FROM t WHERE id = :id');     // user data: ALWAYS prepare
$stmt->execute(['id' => $id]);
$row = $stmt->fetch();          // or false
$all = $stmt->fetchAll();
$n = $stmt->fetchColumn();
$stmt->rowCount();  $pdo->lastInsertId();

$pdo->beginTransaction();  $pdo->commit();  $pdo->rollBack();
```

Debugging

```php
ini_set('display_errors', '1'); error_reporting(E_ALL);
var_dump($x); exit;
error_log(print_r($x, true));          // shows in the php -S terminal
$t = microtime(true); /* ... */ echo microtime(true) - $t;
```

Parse error → look at the line **before**. "Headers already sent" → output (even whitespace) before `header()` / `session_start()` / `setcookie()`.